腾讯 HPC 面试刷题题库

项目深挖・GPU ・Triton ・TP/SP ・分布式通信

面试备战资料

2026-10-21

目录

第一章 使用说明与自测方法 **4**
1.1 题目分级 4
1.2 自测评分 4

第二章 项目深挖题 **5**
2.1 项目概述 5
2.2 瓶颈与优化 5
2.3 结果与可信度 5
2.4 失败与复盘 6

第三章 GPU 架构与性能题 **7**
3.1 执行模型 7
3.2 存储体系 7
3.3 计算与性能模型 7
3.4 优化与分析 8

第四章 Triton 题 **9**
4.1 编程模型 9
4.2 Softmax 与 Reduction 9
4.3 Matmul 与调优 9
4.4 编码与实验 9

第五章 Attention、TP 与 SP 题 **11**
5.1 单卡与 TP 推导 11
5.2 数值白板题 11
5.3 Sequence/Context Parallel 12
5.4 GQA、MQA 与推理 12

第六章 通信与分布式计算题 **13**
6.1 Collective 与公式 13
6.2 拓扑与重叠 13
6.3 并行与扩展 13
6.4 通信计算题 14

第七章 Vibe Coding 题 **15**

第八章 模拟面试套题 **16**

8.1 套题一：基础闭环 16
8.2 套题二：性能深挖 16
8.3 套题三：压力追问 16

第九章 答案要点与评分锚点 18
9.1 项目题答案框架 18
9.2 GPU 题答案要点 19
9.3 Triton 题答案要点 20
9.4 Attention、TP 与 SP 答案要点 21
9.5 通信题答案要点 23
9.6 Vibe Coding 答案要点 24

第十章 最终达标清单 25

第一章　使用说明与自测方法

这份题库面向 GPU 高性能计算、大模型并行与 Triton 岗位。建议先闭卷作答，再到后半部分核对答案要点。

1.1　题目分级

标记	含义	完成标准
P0	本轮面试已经直接涉及	能在白板上完整作答
P1	高频扩展问题	能解释原理、代价和边界
P2	加分题	能给出合理分析路径

1.2　自测评分

每题按 0～4 分评分：

分数	标准
0	完全不会
1	只记得名词
2	能讲概念，缺少公式、shape 或证据
3	能完整回答，并说明关键条件
4	能处理追问、边界、代价和替代方案

建议建立错题表：

题号	首次得分	错误原因	二刷得分	三刷得分

第二章　项目深挖题

这些题没有统一答案。作答时必须使用自己项目的真实数字、Profiler 证据和个人贡献。

2.1　项目概述

1. **P-01** ｜ **P0**：用 60 秒介绍你最重要的项目。
2. **P-02** ｜ **P0**：这个项目解决什么问题？用户或下游系统是谁？
3. **P-03** ｜ **P0**：你个人负责的边界是什么？哪些部分不是你做的？
4. **P-04** ｜ **P0**：典型输入 shape、模型规模、GPU 型号、卡数和互联方式是什么？
5. **P-05** ｜ **P0**：项目最核心的技术难点是什么？

2.2　瓶颈与优化

6. **P-06** ｜ **P0**：优化前的 baseline 是什么？
7. **P-07** ｜ **P0**：你如何判断瓶颈是计算、访存、launch、同步还是通信？
8. **P-08** ｜ **P0**：Nsight Systems 和 Nsight Compute 分别提供了什么证据？
9. **P-09** ｜ **P0**：核心优化为什么有效？减少了哪些 FLOPs、HBM bytes、launch 或等待？
10. **P-10** ｜ **P1**：为什么没有选择另一个方案？
11. **P-11** ｜ **P1**：这个优化在哪些 shape 上无效或变慢？
12. **P-12** ｜ **P1**：如果 GPU 型号更换，结论是否仍成立？

2.3　结果与可信度

13. **P-13** ｜ **P0**：优化前后 latency、throughput、显存和 MFU 分别是多少？
14. **P-14** ｜ **P0**：benchmark 如何 warmup、同步、重复和统计？
15. **P-15** ｜ **P0**：如何验证数值正确性和端到端正确性？
16. **P-16** ｜ **P1**：精度误差阈值为什么合理？
17. **P-17** ｜ **P1**：扩到更多 GPU 后，最先出现的瓶颈是什么？

2.4　失败与复盘

18. **P-18** ｜ **P0**：讲一个失败的优化或错误判断。
19. **P-19** ｜ **P1**：如果重新做一次，你会先测量什么？
20. **P-20** ｜ **P1**：还有哪些优化空间？为什么目前没有继续做？

第三章　GPU 架构与性能题

3.1　执行模型

1. **G-01** ｜ **P0**：画出 GPU、GPC、SM、warp scheduler、CUDA Core、Tensor Core 的层级关系。
2. **G-02** ｜ **P0**：grid、block、warp、thread 分别是什么？block 如何调度到 SM？
3. **G-03** ｜ **P0**：SIMT 与 SIMD 有什么区别？
4. **G-04** ｜ **P0**：什么是 warp divergence？所有分支都会导致明显损失吗？
5. **G-05** ｜ **P1**：GPU 如何用大量 warp 隐藏指令和访存延迟？

3.2　存储体系

6. **G-06** ｜ **P0**：画出 register、shared memory、L1、L2、HBM 的层级、作用域和生命周期。
7. **G-07** ｜ **P0**：什么是 global memory coalescing？举一个好例子和坏例子。
8. **G-08** ｜ **P0**：什么是 shared-memory bank conflict？如何避免？
9. **G-09** ｜ **P0**：register pressure 为什么会影响 occupancy？
10. **G-10** ｜ **P0**：occupancy 越高性能一定越好吗？
11. **G-11** ｜ **P1**：shared memory 一定比 L1 快吗？
12. **G-12** ｜ **P1**：为什么矩阵转置常使用带 padding 的 shared-memory tile？

3.3　计算与性能模型

13. **G-13** ｜ **P0**：CUDA Core 与 Tensor Core 有什么区别？
14. **G-14** ｜ **P0**：什么是 arithmetic intensity？
15. **G-15** ｜ **P0**：写出 Roofline Model，并说明如何判断 compute-bound 和 memory-bound。
16. **G-16** ｜ **P1**：实际性能远低于两条 Roofline 上限时，可能是什么原因？
17. **G-17** ｜ **P0**：为什么 GEMM 使用 tiling 后 arithmetic intensity 会提高？
18. **G-18** ｜ **P0**：为什么小矩阵通常跑不满 GPU？
19. **G-19** ｜ **P1**：fp16、bf16、fp8 在吞吐、动态范围和数值稳定性上如何权衡？
20. **G-20** ｜ **P1**：Ampere 与 Hopper 在大模型计算相关能力上有哪些重要差异？

3.4　优化与分析

21. **G-21** ｜ **P0**：kernel fusion 为什么可能加速?
22. **G-22** ｜ **P1**：为什么融合过多反而可能变慢?
23. **G-23** ｜ **P0**：FlashAttention 为什么省显存、为什么更快?
24. **G-24** ｜ **P0**：FlashAttention 是否减少了 Attention 主项 FLOPs?
25. **G-25** ｜ **P1**：说明 online softmax 需要维护哪些状态。
26. **G-26** ｜ **P0**：Nsight Systems 和 Nsight Compute 的职责有什么不同?
27. **G-27** ｜ **P0**：看到 GPU 时间线有空洞，应按什么顺序排查?
28. **G-28** ｜ **P1**：GPU kernel 是异步执行的，benchmark 应如何计时?
29. **G-29** ｜ **P1**：如何证明一个 kernel 是 memory-bound，而不是凭感觉判断?
30. **G-30** ｜ **P2**：一个 kernel occupancy 下降但速度提高，给出三种合理解释。

第四章　Triton 题

4.1　编程模型

1. **T-01** ｜ **P0**：Triton program instance 与 CUDA block 有什么关系？
2. **T-02** ｜ **P0**：tl.program_id、tl.arange、tl.load、tl.store 分别做什么？
3. **T-03** ｜ **P0**：为什么非整齐 shape 需要 mask？
4. **T-04** ｜ **P1**：mask 是如何参与 load、store 和 reduction 的？
5. **T-05** ｜ **P1**：Triton 相比 CUDA 抽象掉了什么，又没有抽象掉什么？

4.2　Softmax 与 Reduction

6. **T-06** ｜ **P0**：白板写出 row-wise fused softmax。
7. **T-07** ｜ **P0**：softmax 为什么要先减去行最大值？
8. **T-08** ｜ **P0**：padding lane 为什么通常填负无穷？
9. **T-09** ｜ **P1**：为什么 BLOCK_SIZE 常取不小于 n_cols 的 2 的幂？
10. **T-10** ｜ **P1**：一行太长，无法由单个 program 高效处理时怎么办？
11. **T-11** ｜ **P1**：reduction 为什么容易增加 register pressure？

4.3　Matmul 与调优

12. **T-12** ｜ **P0**：给定 C[M,N]=A[M,K]×B[K,N]，解释 BLOCK_M/N/K 的含义。
13. **T-13** ｜ **P0**：num_warps 和 num_stages 分别控制什么？
14. **T-14** ｜ **P1**：为什么 tile 越大不一定越快？
15. **T-15** ｜ **P1**：grouped ordering 为什么可能改善 cache reuse？
16. **T-16** ｜ **P0**：如何设计 autotune 的候选配置和 key？
17. **T-17** ｜ **P0**：一个 Triton kernel 比 PyTorch baseline 慢，如何系统排查？
18. **T-18** ｜ **P1**：如何识别 register spilling？

4.4　编码与实验

19. **T-19** ｜ **P0**：手写 vector add，并正确处理尾块。

20. **T-20** | **P0**：手写 fused softmax，并验证数值稳定性。
21. **T-21** | **P1**：写出 tiled matmul 的核心循环和 accumulator。
22. **T-22** | **P1**：设计 RMSNorm forward，说明累加精度和边界处理。
23. **T-23** | **P1**：为 vector add 推导最小 HBM bytes 和有效带宽。
24. **T-24** | **P1**：设计覆盖连续、非连续、非整齐 shape 和不同 dtype 的测试矩阵。
25. **T-25** | **P2**：一个融合 kernel 在 H=4096 快、H=8192 慢，设计排查实验。

第五章　Attention、TP 与 SP 题

统一符号：

> B 为 batch，S 为 sequence length，H 为 hidden size，A 为 Query head 数，D=H/A，T 为 TP degree。

除非题目另有说明，假设标准 MHA、权重记为 [in_dim,out_dim]、输入为 [B,S,H]、head 数可被 T 整除。

5.1　单卡与 TP 推导

1. **A-01** ｜ **P0**：写出单卡 MHA 从 X 到 Y 的全部 tensor shape。
2. **A-02** ｜ **P0**：QKV projection 的 FLOPs 量级是什么？
3. **A-03** ｜ **P0**：Attention 核心 QK^T 和 Prob×V 的 FLOPs 量级是什么？
4. **A-04** ｜ **P0**：TP 下为什么 QKV 通常使用 column parallel？
5. **A-05** ｜ **P0**：每卡 W_qkv、QKV、Q/K/V 的 shape 是什么？
6. **A-06** ｜ **P0**：为什么按完整 heads 切分后 Attention 核心不需要通信？
7. **A-07** ｜ **P0**：每卡 Score、Prob、Context 的 shape 是什么？
8. **A-08** ｜ **P0**：O projection 为什么使用 row parallel？
9. **A-09** ｜ **P0**：为什么 O projection 后需要求和，而不是 concat？
10. **A-10** ｜ **P0**：普通 TP Attention forward 的关键 collective 在哪里？
11. **A-11** ｜ **P0**：backward 的关键 collective 在哪里？

5.2　数值白板题

使用：

> B=8，S=4096，H=8192，A=64，D=128，T=8，dtype=bf16。

12. **A-12** ｜ **P0**：每卡负责多少个 heads？
13. **A-13** ｜ **P0**：写出每卡 W_qkv 和 QKV 的 shape。
14. **A-14** ｜ **P0**：写出每卡单个 Q/K/V 与 Score 的 shape。
15. **A-15** ｜ **P0**：写出 Context、W_o 和 Y_partial 的 shape。
16. **A-16** ｜ **P0**：若显式保存 Score，每卡需要多少元素和多少 GiB？
17. **A-17** ｜ **P0**：输出 All-Reduce 的逻辑 tensor 有多少元素、多少 MiB？
18. **A-18** ｜ **P1**：8 卡 Ring All-Reduce 按每卡发送量口径需要多少 MiB？

5.3 Sequence/Context Parallel

19. **A-19** | **P0**：经典 Megatron SP 的层间 activation shape 是什么?
20. **A-20** | **P0**：进入 QKV 前做什么 collective，shape 如何变化?
21. **A-21** | **P0**：O projection 后做什么 collective，shape 如何变化?
22. **A-22** | **P0**：经典 Megatron SP 为什么没有直接消除 S^2 Attention activation?
23. **A-23** | **P1**：SP forward 中 All-Gather 和 Reduce-Scatter 在 backward 分别对应什么?
24. **A-24** | **P1**：Ulysses 如何通过 All-to-All 在 sequence shard 与 head shard 之间转换?
25. **A-25** | **P1**：Ring Attention 如何在不聚合完整 K/V 的情况下得到精确结果?

5.4 GQA、MQA 与推理

26. **A-26** | **P0**：GQA 与 MQA 分别是什么?
27. **A-27** | **P1**：KV head 数小于或不能整除 TP degree 时怎么办?
28. **A-28** | **P1**：Decode 阶段本地 Q、KV cache 和 Score 的典型 shape 是什么?
29. **A-29** | **P1**：为什么 Decode 更容易受到 KV cache 带宽和通信 latency 影响?
30. **A-30** | **P2**：比较 Megatron SP、Ulysses 和 Ring/Context Parallel 的通信与显存特征。

第六章　通信与分布式计算题

6.1　Collective 与公式

1. **C-01** ｜ **P0**：解释 All-Reduce、All-Gather、Reduce-Scatter、All-to-All。
2. **C-02** ｜ **P0**：为什么 All-Reduce 可以拆成 Reduce-Scatter + All-Gather?
3. **C-03** ｜ **P0**：写出 α-β 通信模型，并解释两项含义。
4. **C-04** ｜ **P0**：设 tensor 为 N bytes、p 个 rank，写出 Ring AG、RS、AR 的每卡发送量。
5. **C-05** ｜ **P1**：Ring 和 Tree collective 分别更适合什么场景?
6. **C-06** ｜ **P1**：为什么 All-to-All 往往比 Ring All-Reduce 更难优化?

6.2　拓扑与重叠

7. **C-07** ｜ **P0**：NVLink、NVSwitch、PCIe、InfiniBand/RoCE 分别处于什么位置?
8. **C-08** ｜ **P0**：为什么 TP group 通常优先放在节点内?
9. **C-09** ｜ **P1**：rank mapping 为什么会影响性能?
10. **C-10** ｜ **P0**：异步发起通信是否等于成功隐藏通信?
11. **C-11** ｜ **P0**：通信与计算重叠成立需要哪些条件?
12. **C-12** ｜ **P1**：gradient bucket 太大和太小分别有什么问题?
13. **C-13** ｜ **P1**：如何用 Nsight Systems 证明通信真正被隐藏?

6.3　并行与扩展

14. **C-14** ｜ **P0**：DP、TP、PP、SP、CP、EP 分别切分什么?
15. **C-15** ｜ **P1**：经典 Megatron SP 为什么通常不作为独立 world-size 乘数?
16. **C-16** ｜ **P0**：定义 strong scaling 和 weak scaling。
17. **C-17** ｜ **P0**：写出 strong-scaling efficiency。
18. **C-18** ｜ **P1**：GPU 增加后性能扩展变差，列出至少五个原因。
19. **C-19** ｜ **P0**：MFU 是什么？计算时要说明哪些口径?
20. **C-20** ｜ **P1**：MFU 与 HFU 有什么区别?

6.4　通信计算题

21. **C-21** ｜ **P0**：bf16 tensor [8,4096,8192] 有多少 bytes 和 MiB?
22. **C-22** ｜ **P0**：8 卡 Ring All-Reduce 的每卡发送量是多少?
23. **C-23** ｜ **P1**：若有效链路带宽为 200 GB/s，忽略 latency，估算 C-22 的理想时间下限。
24. **C-24** ｜ **P1**：加入每轮 10 μs latency 后，为什么小 tensor 更不利?
25. **C-25** ｜ **P2**：设计一个实验，区分通信带宽瓶颈、collective latency 瓶颈和计算无法隐藏通信。

第七章　Vibe Coding 题

1. **V-01** ｜ **P0**：你如何在实际项目中使用 AI 编程？
2. **V-02** ｜ **P0**：哪些任务适合交给 AI，哪些不适合？
3. **V-03** ｜ **P0**：如何防止 AI 生成不存在的 API 或误解项目约束？
4. **V-04** ｜ **P0**：如何验证 AI 生成的 GPU/Triton kernel？
5. **V-05** ｜ **P1**：AI 声称 benchmark 提升 30%，你会检查什么？
6. **V-06** ｜ **P1**：如何控制大规模修改的风险？
7. **V-07** ｜ **P1**：如何处理密钥、生产数据和公司代码？
8. **V-08** ｜ **P2**：如何评价 “Vibe Coding 会取代底层性能工程师”？

第八章　模拟面试套题

8.1　套题一：基础闭环

建议限时 45 分钟。

1. 60 秒项目介绍。
2. 用 Profiler 证据解释项目的核心瓶颈。
3. 画 GPU memory hierarchy。
4. 解释 coalescing、bank conflict 和 occupancy。
5. 推导 B=8、S=4096、H=8192、A=64、T=8 下的 TP Attention。
6. 加入 Megatron SP 后重新标出 shape 与 collective。
7. 写 Triton softmax 的核心逻辑。
8. 解释 All-Reduce 的 Ring 通信量。
9. 用 30 秒回答 Vibe Coding 的使用原则。

8.2　套题二：性能深挖

建议限时 60 分钟。

1. 讲一个性能优化失败案例。
2. 用 Roofline 分析一个 elementwise kernel 和一个 GEMM。
3. 解释 FlashAttention 的 online softmax。
4. 一个 kernel occupancy 从 75% 降到 50%，但快了 20%，如何分析？
5. Triton matmul 在小 shape 下比 cuBLAS 慢，为什么？
6. 解释 TP forward/backward 的通信对称关系。
7. 比较 Megatron SP、Ulysses 和 Ring Attention。
8. 多机扩展效率突然下降，给出排查树。
9. 设计一套可信的 benchmark。

8.3　套题三：压力追问

建议限时 60 分钟，要求每题继续自问 “为什么” 和 “代价是什么”。

1. 项目提升 30% 的数字可靠吗？逐项证明。

2. 为什么不是 CPU、数据加载或同步瓶颈?
3. 为什么 QKV column parallel 不需要 forward All-Gather?
4. 为什么 O projection 不能 concat?
5. GQA 的 KV heads 少于 TP degree 怎么办?
6. SP 是否一定降低通信量?
7. All-to-All 为什么可能被热点流量拖慢?
8. fusion 为什么可能降低 occupancy?
9. AI 生成的 kernel 正确且更快，是否可以直接上线?
10. 如果只能再准备一天，你会用什么证据证明自己适合这个岗位?

第九章　答案要点与评分锚点

先完成题目，再核对本章。答案是最低覆盖点，不是背诵稿。

9.1　项目题答案框架

9.1.1　P-01～P-05

回答应覆盖：

- 业务或研究问题、规模和用户。
- GPU、卡数、互联、软件栈与典型 shape。
- 个人职责使用 “我定位、我设计、我实现、我验证”。
- 60 秒版本必须包含至少两个数字。

9.1.2　P-06～P-12

回答应形成证据链：

> 端到端时间线 → 热点 kernel/通信 → Roofline 或硬件指标 → 控制变量实验 → 优化方案

不能只说 “GPU 利用率低” 或 “通信慢”。必须说明具体指标、tensor 大小、collective 和测试条件。

9.1.3　P-13～P-17

- baseline 与优化版必须使用相同语义、dtype、shape 和硬件。
- GPU 计时需要 warmup 和正确同步。
- 同时报告 latency、throughput、显存，必要时报告 MFU。
- 正确性至少包含参考实现、误差阈值、边界 shape 和端到端回归。

9.1.4　P-18～P-20

失败案例按 “假设 → 实验 → 证据 → 修正 → 经验” 组织。不要把无关紧要的小问题伪装成失败。

9.2 GPU 题答案要点

9.2.1 G-01～G-05

- 层级：GPU → GPC → SM → warp scheduler 与执行单元。
- 编程层级：grid → block → warp → thread。
- 一个 warp 通常 32 threads；block 被调度到一个 SM。
- SIMT 是多个线程执行共同指令流，但拥有各自状态。
- divergence 主要发生在同一 warp 内走不同路径；warp 间分支一致影响较小。
- latency hiding 依赖足够 ready warps、流水和独立工作。

9.2.2 G-06～G-12

- Register：线程私有、最快但容量有限。
- Shared Memory：block 共享、显式管理。
- L1：每 SM cache；L2：全 GPU 共享；HBM：容量和带宽大、延迟高。
- Coalescing 关注同 warp 地址是否连续、对齐，能否减少 transaction。
- Bank conflict 是同 warp 访问同 bank 不同地址导致串行化；padding 或改索引可缓解。
- register 或 shared-memory 使用过多会减少 SM 上可驻留 blocks/warps。
- occupancy 是隐藏延迟的手段，不是最终性能目标。
- 转置 tile 常增加一列 padding，改变 bank 映射。

9.2.3 G-13～G-20

- Tensor Core 面向矩阵乘累加；CUDA Core 处理通用标量/向量指令。
- Arithmetic Intensity = FLOPs / Bytes。
- Roofline：Performance $\leqslant$ min(Peak Compute, AI $\times$ Memory Bandwidth)。
- 低 AI 倾向 memory-bound，高 AI 且接近峰值时倾向 compute-bound。
- Tiling 让加载的 A/B 元素被多个输出复用，提高 AI。
- 小矩阵可能并行度不足、launch 占比高、tile 利用率低。
- bf16 动态范围接近 fp32，fp16 尾数更多但指数范围小；fp8 吞吐高、需要 scale 和更谨慎的累加。
- Hopper 重点包括更强 Tensor Core、Transformer Engine、TMA 与更丰富的异步流水能力。

9.2.4 G-21～G-30

- Fusion 减少中间 tensor 的 HBM 往返、launch 和同步。
- 过度 fusion 可能增加 register/shared-memory 压力、降低 occupancy、导致 spilling。
- FlashAttention 通过分块和 online softmax 避免完整 S×S score 落 HBM；主要减少 I/O，不消除主项 FLOPs。
- Online softmax 维护行最大值 m、归一化和 l、累积输出 acc，并在新 block 到来时重缩放。
- Nsight Systems 看端到端时间线；Nsight Compute 看单 kernel 硬件指标。

- 正确 benchmark 使用 warmup、CUDA Event 或同步、多次重复和公平 baseline。
- occupancy 下降但变快可能来自 tile 复用提高、指令减少、Tensor Core 利用提高或 HBM bytes 减少。

9.3　Triton 题答案要点

9.3.1　T-01～T-05

- Triton program 通常负责一个数据 tile，编译器再映射到 GPU 执行。
- program_id 确定 tile；arange 生成 tile 内 offsets；load/store 完成向量访存。
- 尾块 mask 防止越界；reduction 的无效 lane 要使用不影响归约的 other 值。
- Triton 抽象部分线程映射和低层调度，但 tile、layout、访存、资源使用仍由开发者负责。

9.3.2　T-06～T-11

Softmax 核心：

```
offsets = arange(0, BLOCK_SIZE)
x = load(row + offsets, mask, other=-inf)
x = x - max(x)
p = exp(x)
y = p / sum(p)
store(y, mask)
```

- 减 max 防止 exp 溢出。
- padding 用负无穷，使其 exp 为 0。
- 2 的幂常有利于编译器映射 reduction。
- 超长行可使用分块、多阶段 reduction 或专用算法。
- reduction 中间值常驻寄存器，宽度增大会提高 register pressure。

9.3.3　T-12～T-18

- 一个 program 计算 BLOCK_M×BLOCK_N 输出 tile，沿 K 以 BLOCK_K 循环。
- num_warps 控制协作 warps；num_stages 控制软件流水深度。
- 大 tile 增加复用，但也增加寄存器、shared memory 和尾块浪费。
- grouped ordering 改变 tile 遍历顺序，提高相邻 program 的 cache reuse。
- autotune 候选应覆盖合理 tile、warps、stages，key 选择会改变最佳配置的 shape 维度。
- 排查顺序：正确性 → 理论 FLOPs/Bytes → layout → tile → 资源 → profiler → 多 shape。
- spilling 可从编译信息、local memory 访问和 profiler 指标识别。

9.3.4 T-19～T-25

- Vector add 最少读取 x、读取 y、写 out，约 3×N×element_size bytes。
- RMSNorm 用 fp32 累加 mean(x^2)，再计算 rsqrt 和缩放。
- 测试必须覆盖非整齐 shape、极小 shape、不同 dtype、连续与非连续输入。
- H=8192 回退优先检查 program 是否过宽、register spilling、occupancy、stage 配置和算法分块边界。

9.4 Attention、TP 与 SP 答案要点

9.4.1 A-01～A-03

单卡：

X [B,S,H]

W_qkv [H,3H] → QKV [B,S,3H]

Q/K/V [B,A,S,D]

Score/Prob [B,A,S,S]

Context [B,A,S,D] → [B,S,H]

W_o [H,H] → Y [B,S,H]

FLOPs：

- QKV projection 约 $6BSH^2$。
- QK^T 约 $2BS^2H$。
- Prob×V 约 $2BS^2H$。
- Attention 核心约 $4BS^2H$。

9.4.2 A-04～A-11

TP 每卡：

W_qkv [H,3H/T]

QKV [B,S,3H/T]

Q/K/V [B,A/T,S,D]

Score/Prob [B,A/T,S,S]

Context [B,S,H/T]

W_o [H/T,H]

Y_partial [B,S,H]

QKV column-parallel forward 不需要通信；完整 heads 可本地完成 Attention。O projection 是沿归约维分片，各卡产生 partial sum，所以 forward 需要 All-Reduce。backward 中 QKV 对 dX 的 partial 需要求和。

9.4.3 A-12～A-18

给定数值：

- 每卡 8 heads。
- W_qkv：[8192,3072]。
- QKV：[8,4096,3072]。
- 单个 Q/K/V：[8,8,4096,128]。
- Score：[8,8,4096,4096]。
- Context：[8,4096,1024]。
- W_o：[1024,8192]。
- Y_partial：[8,4096,8192]。
- Score 元素数：1,073,741,824；bf16 约 2 GiB。
- All-Reduce tensor 元素数：268,435,456；bf16 约 512 MiB。
- 8 卡 Ring All-Reduce 每卡发送约 2×7/8×512=896 MiB。

9.4.4 A-19～A-25

Megatron SP：

X_sp [B,S/T,H]

All-Gather(sequence) → [B,S,H]

本地 head-parallel Attention

Y_partial [B,S,H]

Reduce-Scatter(sequence) → [B,S/T,H]

- Forward AG 的 backward 对应 RS；forward RS 的 backward 对应 AG。
- 经典 SP 在 Attention 内仍持有完整 S，因此不直接缩小 S^2。
- Ulysses 用 A2A 将 [B,S/P,A,D] 转为 [B,S,A/P,D]，完成后反向转换。
- Ring/Context Parallel 保持本地 Q，环形传 K/V block，用 online softmax 合并。

9.4.5 A-26～A-30

- GQA：KV heads 少于 Query heads；MQA：只有一个 KV head。
- KV heads 不能按 T 整除时，可复制 K/V、限制 Attention TP degree 或使用专门映射。

- Decode 常见 Q：[B,A/T,1,D]；KV cache：[B,A_kv/T,S_cache,D]；Score：[B,A/T,1,S_cache]，前提是 KV 可切。
- Decode 单 token 计算量小，更容易暴露 KV cache 带宽、调度和通信 latency。
- Megatron SP 主要降低层间 activation 冗余；Ulysses 依赖 A2A；Ring 依赖 K/V P2P 与计算重叠。

9.5 通信题答案要点

9.5.1 C-01～C-06

- AR：每卡得到完整归约结果。
- AG：各卡分片拼为完整结果。
- RS：先归约，再让每卡保留一片。
- A2A：每卡向不同 peer 发送不同分片。
- AR = RS + AG。
- T_comm ≈ α× 轮次或消息项 + β×bytes。
- Ring：AG 和 RS 每卡约发送 (p-1)/p×N；AR 约 2(p-1)/p×N。
- Ring 对大消息带宽效率较好；Tree 步数 O(log p)，小消息 latency 更有优势。
- A2A 易受并发 peer、拥塞、拓扑和负载不均影响。

9.5.2 C-07～C-13

- 节点内常见 NVLink/NVSwitch 或 PCIe；跨节点使用 NIC 与 InfiniBand/RoCE。
- TP 每层高频通信且位于关键路径，优先映射到节点内高速链路。
- 异步只表示调用不阻塞 CPU，不保证 GPU 时间线上被计算覆盖。
- 重叠需要数据 ready、存在独立计算、独立 stream、正确依赖以及资源不严重争用。
- bucket 太小放大 latency；太大发起太晚。
- 用 Nsight Systems 检查通信区间是否与有效计算重叠，以及关键路径最终等待多少。

9.5.3 C-14～C-20

- DP 切 batch；TP 切权重/hidden/head；PP 切 layers；SP/CP 切 sequence/context；EP 切 experts。
- Megatron SP 常复用 TP group，所以通常不是独立 world-size 乘数。
- Strong scaling 固定总问题规模；weak scaling 固定每卡问题规模。
- Strong efficiency = T_1 / (p×T_p)。
- 扩展变差可能来自通信占比、跨节点链路、小 GEMM、负载不均、pipeline bubble、CPU/数据输入等。
- MFU = 实际模型理论 FLOPs/s ÷ 对应精度硬件峰值 FLOPs/s。
- HFU 可计入重计算等实际执行 FLOPs，因此可能高于 MFU。

9.5.4　C-21～C-25

- [8,4096,8192] bf16：536,870,912 bytes，约 512 MiB。
- 8 卡 Ring AR：约 896 MiB 每卡发送。
- 896 MiB / 200 GB/s 的理想下限约 4.7 ms；需说明 GB 与 GiB 口径，真实时间更高。
- 小 tensor 中固定 latency 和多轮启动占比更高。
- 区分瓶颈可扫描消息大小、rank 数、单机/多机、bucket，并观察带宽曲线与时间线重叠。

9.6　Vibe Coding 答案要点

9.6.1　V-01～V-04

推荐主线：

> 明确契约与验收 → 先读代码复述约束 → 小步修改 → 测试先行 → 审查 diff → 真实执行 → benchmark/profiler → 汇报风险

适合样板代码、测试、benchmark、日志分析和候选实现；并行语义、数值关键路径、安全操作不能无审查托管。

9.6.2　V-05～V-08

- 核对 baseline、shape、dtype、warmup、同步、统计方法与正确性。
- 使用小 diff、小 commit、独立分支或 worktree 控制风险。
- 不提供密钥、生产数据或未经授权的公司代码。
- AI 提高生成和搜索速度，但底层性能工程仍需要硬件模型、并行推导、测量和责任判断。

第十章　最终达标清单

□ P0 项目题平均至少 3 分
□ 能闭卷画 GPU 与 memory hierarchy
□ 能手写 Roofline 公式并分析实例
□ 能手写 Triton vector add 和 softmax
□ 能在 10 分钟内推导完整 TP Attention
□ 能在 TP 上加入 Megatron SP
□ 能解释 Ulysses 与 Ring Attention
□ 能计算 Ring collective 通信量
□ 能用 Profiler 证据解释瓶颈
□ 完成三套模拟题并记录错题